

## 1 Introdução

A Busca em Largura, ou *Breadth-First Search* (BFS), é um dos algoritmos fundamentais de travessia em grafos [5]. O algoritmo parte de um vértice inicial e, em seguida, visita os demais vértices em camadas: primeiro os que estão a uma aresta de distância, depois os que estão a duas arestas, e assim sucessivamente. Dessa forma, em um grafo não ponderado, a BFS calcula a menor distância, em número de arestas, entre o vértice inicial e todos os vértices alcançáveis a partir dele. A Busca em Largura também gera uma árvore chamada *Árvore de Busca em Largura*, na qual o pai de um vértice é aquele que o descobriu durante a busca.

A implementação clássica utiliza uma fila. O vértice inicial é inserido na fila, marcado como visitado, e sua distância é definida como zero. Em seguida, enquanto a fila não estiver vazia, repete-se a seguinte operação: retira-se um vértice  $v$  da fila e examinam-se todos os vértices  $u$  vizinhos de  $v$ . Caso  $u$  ainda não tenha sido visitado, sua distância é calculada, ele é marcado como visitado e adicionado à fila.

Uma forma simplificada do algoritmo é apresentada a seguir:

```
BFS(G, origem):
    para cada vertice v em G:
        pai[v] = none

    pai[origem] = origem
    fila = {origem}

    enquanto fila nao esta vazia:
        u = proximo elemento da fila
        para cada vizinho v de u:
            se pai[v] == none:
                pai[v] = u
                adiciona v na fila
```

## 2 BFS em Fronteiras

O algoritmo de BFS também pode ser implementado como uma descoberta por fronteiras. Inicializa-se a primeira fronteira com o vértice inicial. Em seguida, para cada vértice presente na fronteira atual, examinam-se todos os seus vizinhos; aqueles que ainda não foram visitados são adicionados à próxima fronteira. Esse processo se repete sucessivamente, até que a fronteira atual esteja vazia.

Uma versão simplificada do algoritmo é apresentada a seguir:

```
BFS(G, origem):
    para cada vertice v em G:
        pai[v] = none

    pai[origem] = origem
    fronteira = {origem}

    enquanto fronteira nao esta vazia:
```

```

proxima_frenteira = vazia
para cada vertice u na frenteira:
    para cada vizinho v de u:
        se pai[v] == none:
            pai[v] = u
            adiciona v na proxima_frenteira
frenteira = proxima_frenteira

```

## 2.1 Exemplo Visual de Fronteiras

Considere o grafo da Figura 1. A busca começa no vertice A. Em cada passo, os vertices em azul representam a frenteira atual, os vertices em cinza já foram visitados em níveis anteriores, e os vertices em branco ainda não foram descobertos. As arestas destacadas em azul formam a árvore de BFS construída até aquele momento. Essa árvore registra, para cada vertice descoberto, qual foi o vertice responsável por sua primeira descoberta. Assim, cada aresta azul liga um vertice ao seu pai na busca, permitindo recuperar um caminho mínimo, em número de arestas, da origem A até qualquer vertice alcançável.

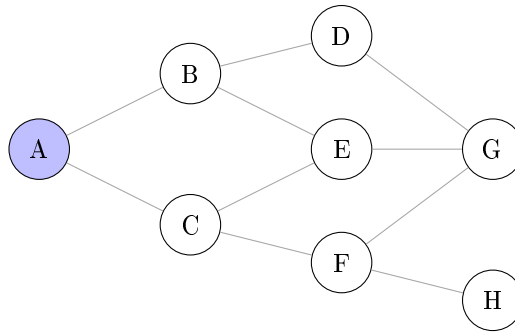


Figure 1: Passo 0 da BFS: a frenteira inicial contém apenas a origem A.

No passo inicial, a distância de A é zero. A primeira expansão examina as arestas que saem de A. Como B e C ainda não foram visitados, ambos são descobertos, recebem distância um e passam a formar a próxima frenteira.

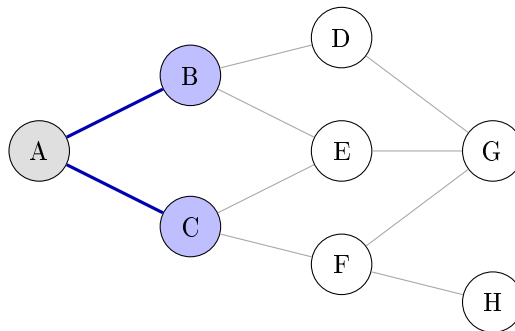


Figure 2: Passo 1 da BFS: B e C formam a fronteira de nível 1.

No segundo passo, a BFS expande todos os vertices da fronteira atual. O vertice B descobre D e E. O vertice C tambem possui uma aresta para E, mas E ja pode ter sido descoberto por B; nesse caso, a implementacao mantem apenas um pai para E. O vertice C tambem descobre F. Assim, a nova fronteira passa a ser formada por D, E e F, todos com distancia dois.

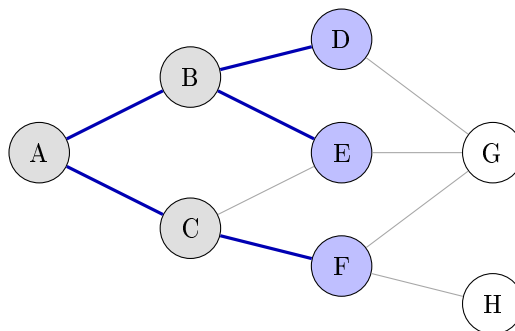


Figure 3: Passo 2 da BFS: D, E e F formam a fronteira de nível 2.

No terceiro passo, a fronteira de nível dois e expandida. Os vertices D, E e F possuem arestas para G. O primeiro que conseguir registrar G como descoberto define seu pai na arvore de BFS. Na figura, E foi escolhido como pai de G, mas D ou F tambem poderiam ser pais validos, pois todos pertencem ao nivel anterior. Alem disso, F descobre H, que tambem passa a fazer parte da fronteira de nivel tres.

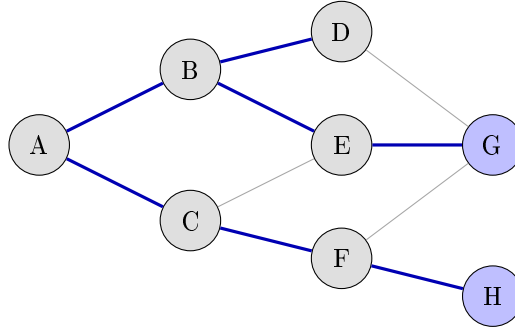


Figure 4: Passo 3 da BFS: G e H são descobertos no nível 3.

Depois da expansão de G, nenhum novo vértice é encontrado. A próxima fronteira fica vazia e o algoritmo termina. O resultado é uma árvore de BFS na qual cada vértice alcançável possui um pai em um nível imediatamente anterior ao seu.

### 3 Divisão do Grafo entre Workers

Em uma implementação paralela ou distribuída, o grafo não precisa ficar inteiro em um único processo. Uma estratégia comum é dividir seus vértices entre vários *workers*. Cada worker fica responsável por armazenar e processar uma parte dos vértices e de suas listas de adjacência. Durante a BFS, quando um worker expande um vértice local, ele examina suas arestas; se o vizinho pertence a outro worker, a descoberta desse vizinho precisa ser comunicada ao worker que o possui.

Uma forma simples de particionamento é a distribuição cíclica. Com quatro workers, o dono de um vértice pode ser calculado por:

$$\text{worker}(v) = v \bmod 4$$

Assim, os vértices são distribuídos de maneira alternada entre os workers. Essa estratégia não garante balanceamento perfeito para todos os grafos, mas é simples e evita concentrar intervalos inteiros de vértices em um único worker. A Tabela 1 mostra um exemplo com 16 vértices.

| Worker | Vértices pertencentes ao worker |
|--------|---------------------------------|
| 0      | 0, 4, 8, 12                     |
| 1      | 1, 5, 9, 13                     |
| 2      | 2, 6, 10, 14                    |
| 3      | 3, 7, 11, 15                    |

Table 1: Particionamento cíclico de 16 vértices entre 4 workers.

A Figura 5 mostra a visão global de um grafo com 16 vértices. Essa visão

representa o grafo completo antes de considerar qual worker é responsável por cada vértice.

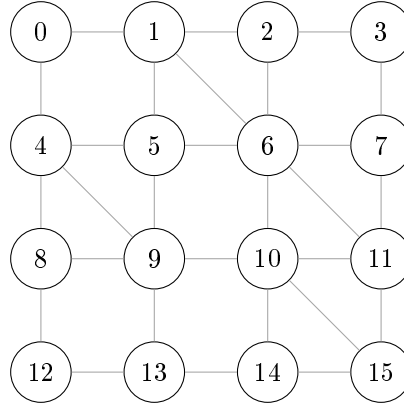


Figure 5: Visão global de um grafo com 16 vértices antes do particionamento.

A Figura 6 mostra o mesmo grafo particionado entre quatro workers. As cores indicam o dono de cada vértice. As arestas que ligam vértices de cores diferentes representam comunicações potenciais durante a BFS, pois a expansão de um vértice em um worker pode descobrir um vértice que pertence a outro worker.

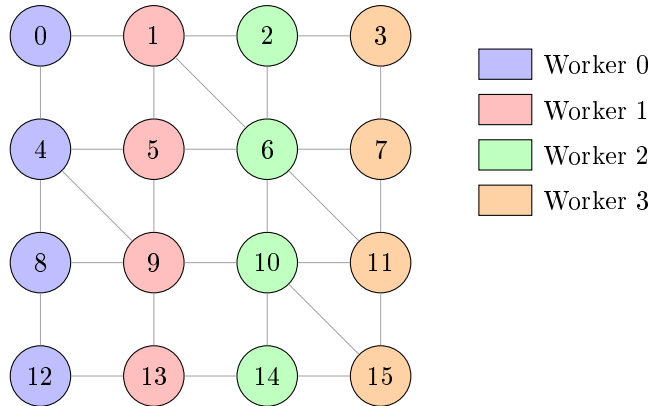


Figure 6: Mesmo grafo particionado entre 4 workers usando a regra  $worker(v) = v \bmod 4$ .

### 3.1 Exemplo de BFS Paralela com 4 Workers

Considere uma BFS iniciada no vértice 0 do grafo particionado da Figura 6. No passo inicial, apenas o worker 0 possui trabalho, pois ele é o dono do vértice 0. Ao expandir 0, são descobertos os vértices 1 e 4. O vértice 4 pertence ao próprio

worker 0, enquanto o vértice 1 pertence ao worker 1. Portanto, a nova fronteira já fica distribuída entre mais de um worker.

Após a expansão dos vértices 1 e 4, a BFS chega ao passo mostrado na Figura 9. Nesse momento, a fronteira global de nível 2 é formada por vértices armazenados em workers diferentes. O worker 0 possui o vértice 8 na fronteira, o worker 1 possui os vértices 5 e 9, o worker 2 possui os vértices 2 e 6, e o worker 3 ainda não possui vértices na fronteira atual.

A expansão passo a passo é mostrada nas figuras seguintes. Em cada uma delas, a fronteira global é dividida entre os quatro workers. Os vértices em azul formam a fronteira local do passo atual, os vértices em cinza já foram visitados, e os vértices em branco ainda não foram descobertos. Depois que todos os workers expandem suas fronteiras locais, os vértices recém-descobertos formam a fronteira do próximo nível.

No passo 0, apenas o worker 0 possui trabalho, pois a origem 0 pertence a ele.

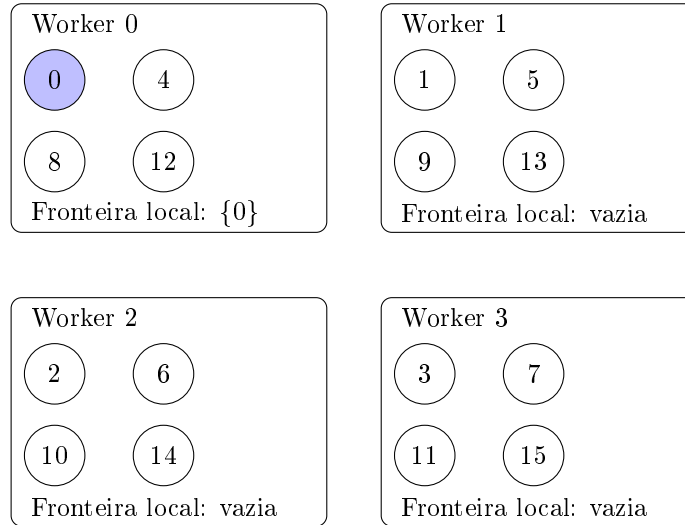


Figure 7: Passo 0 da BFS paralela: apenas a origem 0 está na fronteira.

Ao expandir 0, são descobertos 1 e 4. O vértice 4 permanece no worker 0, e o vértice 1 passa a compor a fronteira local do worker 1.

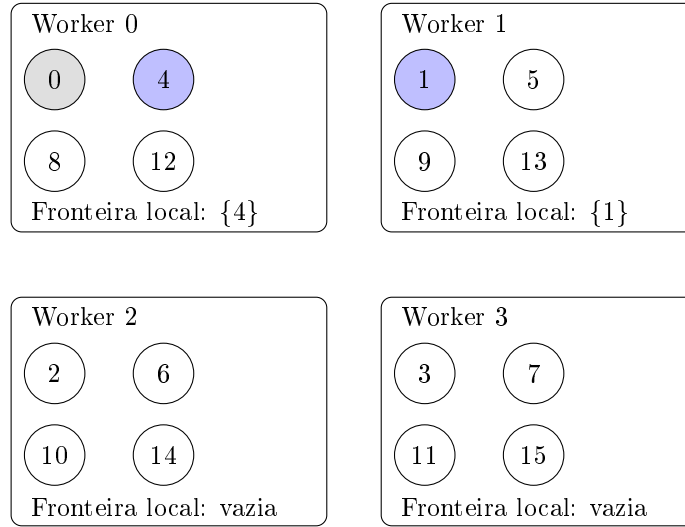


Figure 8: Passo 1 da BFS paralela: os workers 0 e 1 possuem vértices na fronteira.

No passo 1, os workers 0 e 1 trabalham ao mesmo tempo: 4 descobre 8 e 9, enquanto 1 descobre 2, 5 e 6. No passo 2, já há trabalho em três workers diferentes.

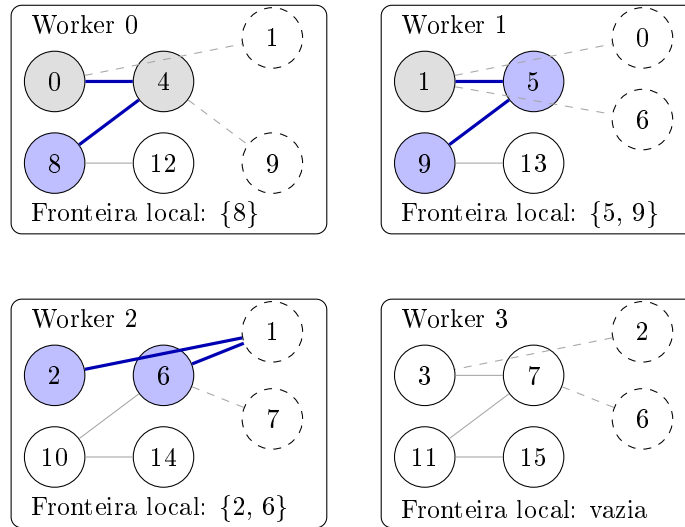


Figure 9: Passo 2 da BFS paralela: a fronteira global está distribuída entre os workers 0, 1 e 2. Vértices tracejados representam vizinhos remotos mantidos apenas para ilustrar comunicações entre workers.

No passo 2, a expansão dos vértices 2 e 6 no worker 2 gera trabalho para o worker 3, pois existem arestas para os vértices 3 e 7. Ao mesmo tempo, os workers 0, 1 e 2 descobrem outros vértices locais, formando a próxima fronteira.

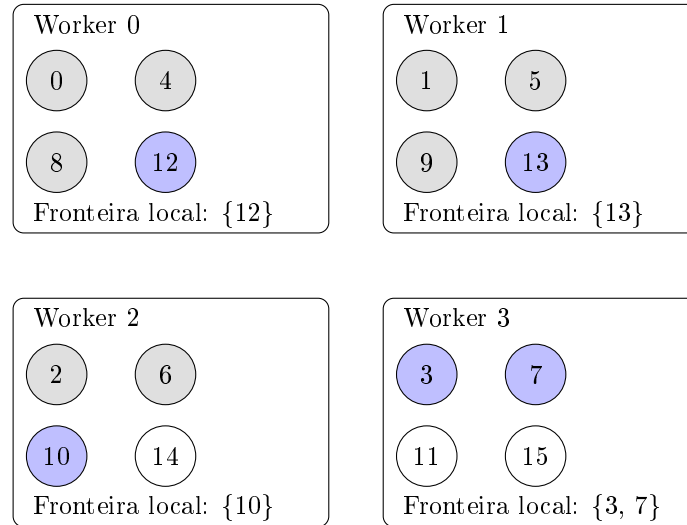


Figure 10: Passo 3 da BFS paralela: todos os workers possuem vértices na fronteira.

Após o passo 3, apenas os workers 2 e 3 continuam com novos vértices a expandir.

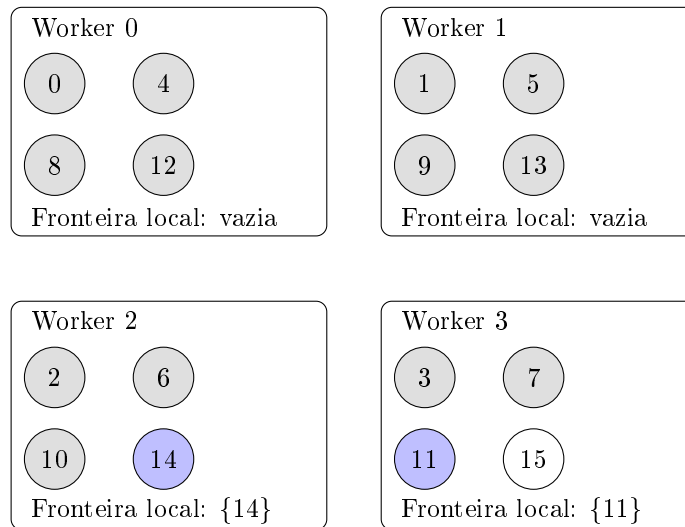


Figure 11: Passo 4 da BFS paralela: apenas os workers 2 e 3 continuam com fronteiras locais.

Por fim, a expansão de 11 ou 14 descobre 15, que pertence ao worker 3. Depois de expandir 15, nenhuma nova descoberta é produzida e a BFS termina.

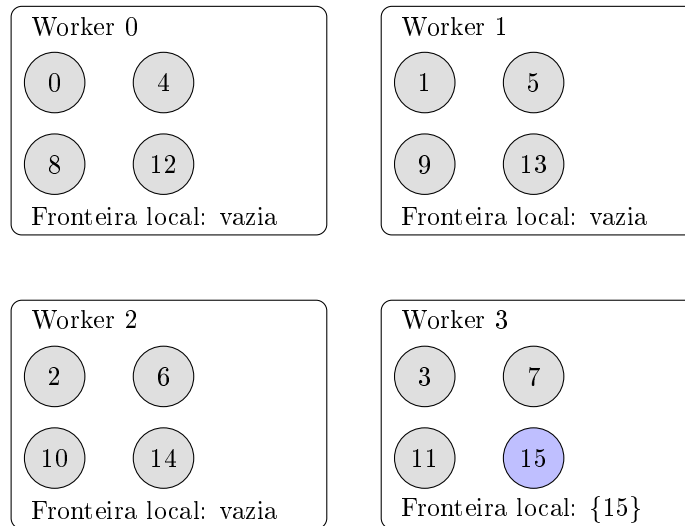


Figure 12: Passo 5 da BFS paralela: o último vértice da fronteira está no worker 3.

Essa sequência visual mostra que a fronteira global da BFS é a união das fronteiras locais de cada worker. Em uma implementação paralela, cada worker ex-

pande apenas os vértices da sua fronteira local. Quando uma aresta aponta para um vértice remoto, a atualização precisa ser enviada ao worker dono daquele vértice, que poderá processá-lo no próximo passo.

## 4 Problemas Reais

Grafos aparecem naturalmente em problemas reais de grande escala. Em uma rede social, vertices podem representar usuarios e arestas podem representar amizades, seguidores, mensagens ou interacoes [6]. Na Web, paginas sao vertices e hiperlinks sao arestas [4]. Em seguranca computacional, grafos podem modelar fluxos de rede, dependencias entre processos, tentativas de acesso e relacoes entre entidades suspeitas [1]. Em biologia, vertices podem representar proteínas, genes ou moléculas, e arestas podem representar interacoes [2]. Em todos esses casos, a BFS aparece como uma primitiva para calcular alcancabilidade, distancia em numero de arestas e expansao de vizinhanca.

A literatura de sistemas de grafos trata a BFS como uma das operacoes centrais para avaliar plataformas de processamento de dados irregulares. O Graph500 foi proposto para complementar benchmarks numericos tradicionais com uma carga de trabalho orientada a dados, motivado pela diferenca entre simulacoes fisicas, que tendem a ser estruturadas e intensivas em ponto flutuante, e aplicacoes de informatica, que tendem a ser nao estruturadas, orientadas a inteiros e com baixa localidade [10]. Sua especificacao define a BFS como o Kernel 2 do benchmark, executado sobre grafos sinteticos de escala crescente, e usa a metrica TEPS, ou arestas atravessadas por segundo, para medir desempenho [7]. Isso mostra que a BFS nao e apenas um exercicio teorico: ela e usada como proxy para medir a capacidade de supercomputadores processarem analise de grafos massivos.

Em redes sociais, a BFS modela consultas de vizinhanca e distancia social. A pergunta “quais usuarios estao a ate dois ou tres saltos de uma conta?” e uma execucao limitada de BFS. Esse tipo de consulta aparece em recomendacao de amizades, propagacao de informacao, deteccao de comunidades locais e analise de influencia [6]. Em grafos com bilhoes de relacoes, a expansao de poucos niveis pode atingir uma quantidade enorme de vertices, especialmente quando existem hubs de grau muito alto. Por isso, mesmo uma BFS limitada por profundidade pode exigir processamento paralelo e estruturas de dados distribuidas.

Em grafos da Web, a BFS tambem e uma abstracao natural. Um rastreador *web crawler* pode iniciar em um conjunto de paginas sementes e expandir links por camadas, descobrindo novas paginas a cada nivel [4]. O sistema Pregel, do Google, foi proposto para processamento de grafos em larga escala e apresenta algoritmos como caminhos minimos, PageRank, propagacao de rotulos e outras computacoes iterativas sobre grafos distribuidos [8]. Embora Pregel seja um modelo mais geral que BFS, sua execucao em superpassos e troca de mensagens entre vertices e muito proxima da ideia de fronteiras sucessivas usada em BFS distribuida.

Em seguranca, investigacao de fraudes e analise de dependencias, BFS e

usada para expandir a vizinhança de uma entidade de interesse. A partir de uma conta, um endereço IP, uma máquina infectada ou um processo suspeito, a busca em largura identifica entidades conectadas por poucos saltos. Essa forma de exploração é importante porque muitas relações relevantes não aparecem em uma aresta direta, mas em cadeias curtas: contas que compartilham dispositivos, máquinas que se comunicam com o mesmo servidor, ou processos que dependem dos mesmos arquivos [1]. Em grafos corporativos ou de telecomunicações, essas buscas podem envolver centenas de milhões de vértices e precisam responder em tempo interativo ou quase interativo.

O desafio é que os grafos modernos podem ser massivos e irregulares. Grafos de redes sociais e da Web frequentemente possuem distribuição de grau altamente desbalanceada: poucos vértices concentram muitas arestas, enquanto a maioria tem grau baixo. Essa propriedade gera duas dificuldades para BFS. Primeiro, há desbalanceamento de trabalho, pois expandir um vértice de alto grau custa muito mais do que expandir um vértice comum. Segundo, há muita contenção, pois muitos caminhos podem tentar descobrir o mesmo vértice ao mesmo tempo.

Além disso, a BFS tem baixa localidade de memória. Em matrizes densas ou vetores, os acessos costumam ser contíguos e previsíveis. Em grafos, ao expandir a fronteira, o algoritmo segue listas de adjacência que podem apontar para posições distantes e aparentemente aleatórias. Consequentemente, a BFS é normalmente limitada por largura de banda, latência de memória, atômicos, sincronização e comunicação entre máquinas, e não por operações aritméticas.

Trabalhos em arquiteturas paralelas mostram como essas dificuldades aparecem na prática. Beamer, Asanovic e Patterson propuseram a BFS *direction optimizing*, que alterna entre uma expansão top-down, baseada na fronteira atual, e uma expansão bottom-up, que examina vértices ainda não visitados procurando vizinhos na fronteira [3]. Essa mudança reduz trabalho em grafos de pequeno diâmetro e grande fronteira, um caso comum em grafos sociais e da Web. Merrill, Garland e Grimshaw estudaram BFS em GPU e mostraram que a primitiva é representativa de computações paralelas com acesso a memória e distribuição de trabalho irregulares; sua implementação alcançou bilhões de arestas atravessadas por segundo em uma ou mais GPUs [9].

Portanto, BFS aparece em problemas reais de duas formas complementares. Como algoritmo final, ela responde consultas de alcance, distância e vizinhança. Como primitiva de sistema, ela estressa exatamente os componentes que tornam grafos massivos difíceis: memória irregular, fronteiras dinâmicas, atômicos, desbalanceamento e comunicação. Por isso, estudar BFS em GPU e em ambientes distribuídos é uma forma direta de estudar o desempenho de análise de grafos em escala real.

## 5 Motivação de Paralelismo

A BFS possui uma oportunidade natural de paralelismo: todos os vértices de uma mesma fronteira podem ser processados simultaneamente. Como eles per-

tencem ao mesmo nível da busca, a expansão de suas arestas pode ocorrer em paralelo desde que a implementação controle corretamente a descoberta de vértices repetidos. Se dois ou mais trabalhadores encontram o mesmo vizinho ao mesmo tempo, apenas um deles deve registrar esse vértice como descoberto e inseri-lo na próxima fronteira.

Essa propriedade torna a BFS adequada para GPUs, que são projetadas para executar milhares de threads leves em paralelo. Em uma GPU, diferentes threads podem examinar vértices ou arestas distintas da fronteira. Quando a fronteira é grande, o algoritmo pode expor paralelismo suficiente para ocupar muitos multiprocessadores da GPU. Como a BFS realiza pouca computação por aresta, o principal ganho esperado vem da capacidade da GPU de sustentar muitas operações de memória simultâneas e esconder parte da latência por meio de concorrência.

No entanto, paralelizar BFS em GPU não é trivial. A primeira dificuldade é o desbalanceamento de carga. Em grafos com distribuição de grau irregular, uma thread responsável por um vértice de grau muito alto pode ter muito mais trabalho do que outra thread responsável por um vértice de grau baixo. Por isso, implementações eficientes geralmente distribuem o trabalho por arestas, por grupos de threads ou por blocos, em vez de usar sempre uma thread por vértice.

A segunda dificuldade é a construção da próxima fronteira. Muitas threads podem descobrir novos vértices ao mesmo tempo, então a estrutura que armazena a fronteira precisa aceitar inserções concorrentes. Isso normalmente exige operações atômicas, como incremento atômico de um contador de tamanho ou compare-and-swap no vetor de pais. Essas operações garantem correção, mas podem causar contenção quando muitas threads tentam atualizar a mesma região de memória.

A terceira dificuldade aparece em ambientes distribuídos. Quando o grafo não cabe em uma única GPU, vértices e arestas precisam ser particionados entre vários processos ou dispositivos. Durante a expansão da fronteira, uma aresta pode apontar para um vértice pertencente a outro processo. Nesse caso, a descoberta precisa ser comunicada ao proprietário do vértice remoto. Como o padrão de comunicação depende dos dados do grafo e muda a cada nível da BFS, essa comunicação é irregular e difícil de otimizar com mensagens grandes e previsíveis.

Portanto, a motivação para usar paralelismo em GPU é dupla. Primeiro, a BFS precisa de alto paralelismo para processar grandes volumes de arestas em tempo razoável. Segundo, sistemas modernos de GPU oferecem alta largura de banda de memória e interconexões rápidas entre dispositivos, características importantes para algoritmos de grafos massivos. O desafio de pesquisa é organizar o trabalho e a comunicação de forma alinhada ao hardware.

## 6 Implementacao GPU side em NVSHMEM

NVSHMEM e uma biblioteca de comunicacao para GPUs baseada no modelo de espaco de enderecamento global particionado. Nesse modelo, cada elemento de processamento, ou PE, possui sua memoria local, mas tambem pode acessar regioes simetricas de memoria de outros PEs por meio de operacoes de comunicacao unilaterais. A diferenca importante para este trabalho e que essas operacoes podem ser chamadas diretamente de kernels CUDA, isto e, pelo proprio codigo que esta executando na GPU.

Em uma BFS distribuida, essa capacidade e relevante porque a comunicacao remota e descoberta dinamicamente. Ao processar uma aresta  $(u, v)$ , uma thread da GPU pode determinar que  $v$  pertence a outro PE. Com NVSHMEM, essa thread pode tentar atualizar diretamente o estado remoto de  $v$ , sem precisar retornar ao CPU para empacotar uma mensagem MPI naquele momento. Isso reduz a dependencia de comunicacao mediada pelo host e aproxima o mecanismo de comunicacao do ponto onde a informacao e produzida.

Uma estrategia comum e particionar os vertices de forma ciclica entre os PEs. O proprietario de um vertice  $v$  pode ser calculado como  $v \% \text{num\_pes}$ , e o indice local como  $v / \text{num\_pes}$ . Assim, qualquer thread consegue descobrir rapidamente qual PE e responsavel por armazenar o estado de um vertice. O vetor de pais, o vetor de distancias e as estruturas de fronteira podem ser alocados em memoria simetrica, permitindo atualizacoes locais ou remotas por NVSHMEM.

O passo principal da BFS em GPU consiste em expandir a fronteira atual. Cada thread examina uma parte das arestas dos vertices ativos. Para cada vizinho, a thread verifica se ele ainda nao foi descoberto. Em uma implementacao concorrente, essa verificacao precisa ser atomica. Um compare-and-swap sobre o vetor de pais pode trocar o valor inicial, por exemplo  $-1$ , pelo pai que descobriu o vertice. Se a troca for bem-sucedida, a thread sabe que foi a primeira a descobrir aquele vertice e pode inserir o vizinho na proxima fronteira.

Quando o vizinho pertence ao mesmo PE, a operacao atomica pode ser local. Quando pertence a outro PE, a mesma ideia pode ser expressa como uma operacao atomica remota em NVSHMEM. A operacao remota tenta reivindicar o vertice no PE proprietario. Esse padrao e adequado para BFS porque a competicao entre varias descobertas do mesmo vertice e uma corrida benigna: nao e necessario que o primeiro thread em tempo real seja o pai definitivo, apenas que o pai escolhido esteja no nivel anterior e que a arvore resultante seja valida.

Um esboco de expansao de fronteira com comunicacao do lado da GPU e mostrado a seguir:

```
for each active vertex u in current_frontier, in parallel:
    for each neighbor v of u:
        owner = v % num_pes
        local_index = v / num_pes

        old = atomic_compare_and_swap(parent[owner][local_index], -1, u)
        if old == -1:
```

`insert v into next_frontier of owner`

Na pratica, a implementacao precisa tratar detalhes adicionais. A insercao na proxima fronteira pode exigir contadores atomicos locais ou remotos. Tambem e necessario sincronizar os PEs entre niveis para garantir que todas as descobertas do nivel atual foram completadas antes do inicio do proximo nivel. Ao final, a BFS termina quando todas as fronteiras locais estao vazias e nenhum PE possui trabalho pendente.

O uso de NVSHMEM se alinha bem ao comportamento irregular da BFS. Em vez de forçar a GPU a acumular descobertas, sincronizar com o CPU e entao realizar uma troca de mensagens, a propria thread que encontra uma aresta remota pode emitir a atualizacao. Esse modelo nao elimina os custos de comunicacao, atomicos e sincronizacao, mas reduz a distancia entre computacao e comunicacao. Para grafos massivos, onde a quantidade de arestas visitadas e grande e o padrao de destinos e imprevisivel, essa aproximacao pode melhorar a escalabilidade e a utilizacao do hardware.

No contexto desta tese, a BFS em NVSHMEM representa uma abordagem *GPU-side*: o CPU continua importante para inicializacao, lancamento dos processos, configuracao MPI/NVSHMEM e validacao, mas o laço principal de traversal e comunicacao fina ocorre dentro da GPU. Essa separacao permite usar MPI para coordenacao global e NVSHMEM para atualizacoes remotas de baixa granularidade, combinando dois modelos de programacao adequados a camadas diferentes do sistema.

## References

- [1] Leman Akoglu, Hanghang Tong, and Danai Koutra. Graph based anomaly detection and description: A survey. *Data Mining and Knowledge Discovery*, 29(3):626–688, 2015.
- [2] Albert-Laszlo Barabasi and Zoltan N. Oltvai. Network biology: Understanding the cell’s functional organization. *Nature Reviews Genetics*, 5(2):101–113, 2004.
- [3] Scott Beamer, Krste Asanovic, and David Patterson. Direction-optimizing breadth-first search. In *Proceedings of the International Conference on High Performance Computing, Networking, Storage and Analysis*. IEEE Computer Society Press, 2012.
- [4] Sergey Brin and Lawrence Page. The anatomy of a large-scale hypertextual web search engine. In *Proceedings of the Seventh International Conference on World Wide Web*, pages 107–117. Elsevier Science Publishers B. V., 1998.
- [5] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009.

- [6] David Easley and Jon Kleinberg. *Networks, Crowds, and Markets: Reasoning About a Highly Connected World*. Cambridge University Press, 2010.
- [7] Graph 500 Steering Committee. Graph 500 Benchmark Specification: Graph 500 Benchmarks 1 (Search) and 2 (Shortest Path), 2017. Acesso em: 25 jun. 2026.
- [8] Grzegorz Malewicz, Matthew H. Austern, Aart J. C. Bik, James C. Dehnert, Ilan Horn, Naty Leiser, and Grzegorz Czajkowski. Pregel: A system for large-scale graph processing. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data*, pages 135–146. ACM, 2010.
- [9] Duane Merrill, Michael Garland, and Andrew Grimshaw. Scalable gpu graph traversal. In *Proceedings of the 17th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, pages 117–128. ACM, 2012.
- [10] Richard C. Murphy, Kyle B. Wheeler, Brian W. Barrett, and James A. Ang. Introducing the graph 500. In *Cray User’s Group*. Sandia National Laboratories, may 2010. SAND2010-3263C. Acesso em: 25 jun. 2026.